第5课：Transformers和大语言模型

- 李伟昌, weichangli@zju.edu.cn
- 浙江大学**海洋人工智能**课程讲义
- 版本 1, 2026年5月20日

时长： 45分钟
目标： 理解Transformer架构、自注意力机制和现代LLM

5.1 为什么用Transformers?

RNN的局限性

1. **顺序计算：** 无法并行化（必须从左到右处理）
2. **长距离依赖：** 信息必须通过许多时间步传播
3. **梯度消失：** 即使LSTM也难以处理非常长的序列

Transformer解决方案

- **无循环：** 并行处理整个序列
- **自注意力：** 任意位置之间的直接连接
- **可扩展到：** 数亿/数十亿参数

历史影响

- 2017："Attention Is All You Need"（Google）
- 2018：BERT（双向编码器）
- 2019+：GPT-2、GPT-3、GPT-4（缩放定律）
- 现在：NLP的主导架构，扩展到视觉、音频、多模态

5.2 自注意力机制

核心思想

每个位置同时关注所有其他位置。

Query、Key、Value类比

信息检索隐喻：

- **Query：** 我在寻找什么?
- **Key：** 我包含什么?
- **Value：** 我提供什么信息?

对于每个位置，计算：

1. **Query向量：** $Q = XW_Q$
2. **Key向量：** $K = XW_K$
3. **Value向量：** $V = XW_V$

其中$X \in \mathbb{R}^{n \times d_{model}}$是输入序列。

缩放点积注意力

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

维度：

- $Q \in \mathbb{R}^{n \times d_k}$
- $K \in \mathbb{R}^{n \times d_k}$
- $V \in \mathbb{R}^{n \times d_v}$
- 输出：$\mathbb{R}^{n \times d_v}$

视觉计算流程：

```
第1步：计算相似度分数
         Q (seq_len × d_k)
           │
           ▼
    K^T ───►[×]──────► QK^T (seq_len × seq_len)
   (d_k × seq_len)       注意力分数
                          "每个词对其他词
                           的关注程度"

第2步：缩放和Softmax
    QK^T ───►[/ √d_k]───►[softmax]───► 注意力权重
                          (逐行)        (每行和为1)

第3步：值的加权和
    注意力 ───►[× V]──────► 输出
    权重                     (值的加权和)
```

为什么用$\sqrt{d_k}$缩放?

- 大的d_k → 点积大小增长
- 大值将softmax推入梯度小区域
- 缩放防止这种情况（"softmax饱和"）

注意力矩阵可视化

```
        "The" "cat" "sat" "on" "the" "mat"
"The"   [0.3   0.2   0.2   0.1   0.1   0.1]
"cat"   [0.2   0.4   0.2   0.1   0.1   0.0]
"sat"   [0.1   0.2   0.3   0.2   0.1   0.1]
...
```

每行显示一个词对其他词的关注程度。

PyTorch实现

```python
import torch
import torch.nn as nn
import math

class SelfAttention(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.scale = math.sqrt(d_model)

    def forward(self, x, mask=None):
        # x: (batch, seq_len, d_model)
        Q = self.W_q(x)  # (batch, seq_len, d_model)
        K = self.W_k(x)
        V = self.W_v(x)

        # 注意力分数
        scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale
        # scores: (batch, seq_len, seq_len)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attn_weights = torch.softmax(scores, dim=-1)
        output = torch.matmul(attn_weights, V)

        return output, attn_weights
```

5.3 多头注意力

动机

不同的头可以学习不同类型的关系：

- 头1：句法依赖
- 头2：共指
- 头3：语义相似性

实现

并行运行多个注意力机制，然后拼接：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

每个头：

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

维度： 如果$d_{model} = 512$和$h = 8$头，每个头有$d_k = d_v = 64$。

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        batch_size, seq_len, _ = x.shape

        # 线性投影并重塑为多头的多头
        Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)

        # 注意力
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attn = torch.softmax(scores, dim=-1)
        context = torch.matmul(attn, V)

        # 拼接头
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.d_model)

        return self.W_o(context)
```

5.4 位置编码

问题

自注意力是**置换不变的**："dog bites man"和"man bites dog"产生相同的注意力模式，没有位置信息。

正弦位置编码

原始Transformer使用固定正弦函数：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

特性：

- 每个位置有唯一编码
- 可以外推到更长序列
- 允许模型学习相对位置

学习的位置嵌入

现代方法：像词嵌入一样学习嵌入。

```python
# 正弦版本
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

# 学习版本
self.pos_embedding = nn.Embedding(max_seq_len, d_model)
```

最新： 现代LLM中的旋转位置嵌入（RoPE）。

5.5 完整Transformer架构

编码器块

```
输入 → 多头注意力 → 加 & 归一 → 前馈 → 加 & 归一 → 输出
         ↑                        ↑
     (残差 + 层归一)          (残差 + 层归一)
```

组件：

1. **多头自注意力：** 跨序列混合信息
2. **前馈网络：** $\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$
3. **残差连接：** $x + \text{Sublayer}(x)$
4. **层归一化：** 跨特征归一化

```python
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model)
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # 带残差的自注意力
        attn_out = self.self_attn(x, mask)
        x = self.norm1(x + self.dropout(attn_out))

        # 带残差的前馈
        ff_out = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_out))

        return x
```

解码器块

额外的掩码自注意力（防止看到未来token）+ 到编码器的交叉注意力。

视觉：完整Transformer编码器堆栈

```
输入Token: [The] [cat] [sat] [on] [the] [mat]
              │    │     │    │    │     │
              └────┴─────┴────┴────┴─────┘
                         │
                   ┌─────▼─────┐
                   │ 嵌入      │
                   │  + PosEnc │
                   └─────┬─────┘
                         │ d_model=512
                         ▼
             ┌───────────────────────┐
             │  编码器层 1           │
             │ ┌───────────────────┐ │
             │ │ 多头              │ │
             │ │ 自注意力          │ │
             │ │  (8头)            │ │
             │ └───────┬───────────┘ │
             │         │             │
             │    [加 & 归一]        │
             │         │             │
             │ ┌───────▼───────────┐ │
             │ │ 前馈              │ │
             │ │ (2048 → 512)      │ │
             │ └───────┬───────────┘ │
             │         │             │
             │    [加 & 归一]        │
             │         │             │
             └─────────┬─────────────┘
                       │
             ┌─────────▼─────────────┐
             │  编码器层 2           │  × N (通常6或12)
             └─────────┬─────────────┘
                       │
                      ...
                       │
             ┌─────────▼─────────────┐
             │  编码器层 N           │
             └─────────┬─────────────┘
                       │
                  输出表示
                  (上下文化嵌入)
```

完整编码器（代码）

```python
class TransformerEncoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads, num_layers, d_ff, max_len=5000):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, max_len)
        self.layers = nn.ModuleList([
            TransformerEncoderLayer(d_model, num_heads, d_ff)
            for _ in range(num_layers)
        ])

    def forward(self, x, mask=None):
        x = self.embedding(x)
        x = self.pos_encoding(x)
        for layer in self.layers:
            x = layer(x, mask)
        return x
```

5.6 BERT：双向编码器

架构

- 仅使用Transformer**编码器**
- 双向：关注左右上下文
- Base：12层，768隐藏，110M参数
- Large：24层，1024隐藏，340M参数

预训练任务

1. 掩码语言建模（MLM）：

- 随机掩码15%的token
- 预测原始token
- 示例："The [MASK] sat on the mat" → "cat"

2. 下一句预测（NSP）：

- 给定两个句子，预测B是否跟随A
- 帮助学习句子级关系

微调

预训练BERT → 任务特定头：

- 分类：[CLS] token输出 → 分类器
- NER：每个token输出 → 每token分类器
- QA：开始/结束跨度预测

```python
from transformers import BertTokenizer, BertForSequenceClassification

# 加载预训练
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)

# 分词
text = "This movie was great!"
inputs = tokenizer(text, return_tensors='pt', padding=True, truncation=True)
outputs = model(**inputs)
predictions = torch.argmax(outputs.logits, dim=-1)
```

5.7 GPT：自回归模型

架构

- 仅使用Transformer**解码器**（掩码注意力）
- 单向：只关注过去的token
- 生成性：自回归预测下一个token

训练目标

最大化下一个token的似然：

$$\mathcal{L} = \sum_i \log P(x_i | x_{<i}; \theta)$$

GPT演进

模型	年份	参数	上下文	关键特性
GPT-1	2018	117M	512	无监督预训练
GPT-2	2019	1.5B	1024	零样本能力
GPT-3	2020	175B	2048	上下文学习
GPT-4	2023	?	128K	多模态、推理

缩放定律

性能随以下因素可预测地缩放：

- 模型大小（参数）
- 数据集大小（token）
- 计算（FLOPs）

涌现能力在规模上出现。

5.8 现代LLM技术

关键架构改进

层归一化位置：

- 原始：Post-LN（残差后）
- 现代：Pre-LN（前，更稳定）

激活函数：

- ReLU → GELU（更平滑）

位置编码：

- 正弦 → 旋转（RoPE）
- 允许更好的长度外推

效率技术

Flash Attention： 重新排序计算以减少HBM读取

量化：

- FP32 → FP16/BF16 → INT8 → INT4
- 减少内存并提高速度

KV缓存： 在生成期间存储键/值向量以避免重新计算

5.9 实验：使用Transformers

练习1：实现Mini Transformer

完成Transformer编码器实现：

- 添加适当的掩码
- 在简单任务上测试（例如，序列分类）
- 可视化注意力权重

练习2：微调BERT（20分钟）

为情感分析微调BERT：

```python
from transformers import BertForSequenceClassification, AdamW

model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)
optimizer = AdamW(model.parameters(), lr=2e-5)

# 训练循环
for batch in dataloader:
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
```

练习3：使用GPT生成文本

使用Hugging Face transformers进行生成：

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer

model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

prompt = "Once upon a time"
inputs = tokenizer(prompt, return_tensors='pt')
outputs = model.generate(**inputs, max_length=50, temperature=0.7)
print(tokenizer.decode(outputs[0]))
```

关键要点

1. 自注意力支持并行处理和直接长距离连接
2. 多头注意力允许建模多样化关系
3. 位置编码提供序列顺序信息
4. BERT：用于理解任务的双向编码器
5. GPT：用于生成任务的自回归解码器
6. 缩放定律驱动现代LLM发展
7. 预训练 + 微调是主导范式

延伸阅读

核心论文

1. Attention Is All You Need - 原始Transformer论文（Vaswani等人，2017）
2. The Illustrated Transformer - Jay Alammar的Transformer视觉指南
3. BERT Paper - 深度双向Transformers的预训练
4. Language Models are Few-Shot Learners - GPT-3论文

视频资源

5. **Andrej Karpathy的Neural Networks: Zero to Hero** - 第6讲涵盖Transformers并从头构建GPT。这些笔记中的自注意力计算流程图遵循Karpathy的micrograd风格逐步分解方法。

6. **Andrej Karpathy的Let's build GPT: from scratch, in code, spelled out** - Zero to Hero transformer讲座的配套视频，Karpathy在其中实现字符级GPT。强烈推荐用于理解如何从头实现Transformers。

工具和文档

7. Hugging Face Transformers Documentation - 生产就绪的Transformer实现